ARCHITECTURE

Helix Cluster OS — Hardened Architecture & Component Map

> **HXC-1424.** This document is the canonical hardened architecture diagram and component map. Every component named below links to its implementing package (a real directory in this repository) and its originating gap / work-item ID. The mapping is mechanically enforced: the doc-lint `pkg/archlint` parses the **Component Map** table and fails the build if any mapped component points at a package path that does not exist on disk. A component therefore cannot be documented here without a real implementing package.

Seven-layer stack (L0–L7)

Helix is organised as a seven-layer stack over a hardware substrate (L0), coordinated via **SWIM gossip** for membership and **Raft** consensus for strongly-consistent state, with an **Omega-model** two-level scheduler using optimistic concurrency.

```
┌─────────────────────────────────────────────────────────────────
│ L7  Federation, Multi-cluster &
Observability                                 │
│     federation · fedtopology · metrics(+tier/cost/provider) ·
tracing      │
├─────────────────────────────────────────────────────────────────
│ L6  Control-plane Services (14
microservices)                                │
│     gateway · session · health · policy · llm · build · advisory
· trust   │
├─────────────────────────────────────────────────────────────────
│ L5  Secure Transport &
Networking                                        │
│     hybridkex(X25519+ML-KEM-768) · e2ee-proxy · wireguard · ice
· hashslot │
├─────────────────────────────────────────────────────────────────
│ L4  Scheduling, Placement &
Economics                                      │
│     scheduler(Omega) · constraints · preempt · workloadrouter ·
carbonsched│
│     · marketplaceadapter · economics ·
```

```
revenueopt                                    │
├──────────────────────────────────────────────────────────────────
│ L3  Consensus & Replicated
State                                                        │
│     voting · mvcc · crdt · deltacrdt · antientropy ·
hlc                         │
├──────────────────────────────────────────────────────────────────
│ L2  Membership &
Discovery                                                          │
│     swim · discovery · scan · nattraversal ·
cellmesh                        │
├──────────────────────────────────────────────────────────────────
│ L1  Node lifecycle &
Boot                                                         │
│     node · console · helix-node ·
local(TCO)                                    │
├──────────────────────────────────────────────────────────────────
│ L0  Hardware / Resource
Substrate                                                │
│     resources · gpuattest · gpu · devicecatalog · capability ·
exportcontrol│
└──────────────────────────────────────────────────────────────────
```

Each layer consumes only the layers beneath it. Cross-cutting concerns (attestation, metrics, compliance) are surfaced as dedicated components rather than threaded implicitly through every layer.

Component Map

The table is the machine-readable contract enforced by `pkg/archlint`. The **Package** column is a repository-relative path to the implementing Go package (or submodule root). The **Origin** column records the HXC work-item / gap that introduced or hardened the component.

Component	Layer	Package	Origin
Resource probe & sampling	L0	`pkg/resources`	HXC-1153
GPU attestation crypto	L0	`pkg/gpuattest`	HXC-1434/1573
GPU manager	L0	`internal/gpu`	HXC-1447
Device taxonomy catalogue	L0	`pkg/devicecatalog`	HXC-1331
Capability negotiation	L0	`pkg/capability`	HXC-1305
Export-control KYC gate	L0	`pkg/exportcontrol`	HXC-1482
Node agent	L1	`internal/node`	HXC-1159
Operator console / boot	L1	`internal/console`	HXC-1147
Node binary	L1	`cmd/helix-node`	HXC-1159
Local TCO accounting	L1	`pkg/local`	HXC-1583

Component	Layer	Package	Origin
SWIM gossip membership	L2	`pkg/swim`	HXC-1344
Service discovery	L2	`pkg/discovery`	HXC-1363
SCAN stable endpoint	L2	`pkg/scan`	HXC-1403
NAT traversal (STUN)	L2	`pkg/nattraversal`	HXC-1383
Cell mesh	L2	`pkg/cellmesh`	HXC-1352
Largest-subcluster voting	L3	`pkg/voting`	HXC-1135
MVCC time-travel store	L3	`pkg/mvcc`	HXC-1238
CRDT state	L3	`pkg/crdt`	HXC-1240
Delta-state CRDTs	L3	`pkg/deltacrdt`	HXC-1409
Anti-entropy repair	L3	`pkg/antientropy`	HXC-1241
Hybrid logical clock	L3	`pkg/hlc`	HXC-1239
Omega scheduler	L4	`pkg/scheduler`	HXC-1137
Scheduler service	L4	`internal/scheduler`	HXC-1137
Placement constraints	L4	`pkg/constraints`	HXC-1136
Preemption	L4	`pkg/preempt`	HXC-1136
Workload router	L4	`pkg/workloadrouter`	HXC-1455
Carbon-aware placement	L4	`pkg/carbonsched`	HXC-1480
Marketplace adapter	L4	`pkg/marketplaceadapter`	HXC-1454
Reward economics	L4	`pkg/economics`	HXC-1460/1461
Revenue optimiser	L4	`pkg/revenueopt`	HXC-1456
Hybrid PQ key exchange	L5	`pkg/hybridkex`	HXC-1476
E2EE proxy	L5	`cmd/e2ee-proxy`	HXC-1532
WireGuard mesh	L5	`internal/wireguard`	HXC-1163
ICE connectivity	L5	`pkg/ice`	HXC-1349
Hash-slot routing	L5	`pkg/hashslot`	HXC-1135
Gateway service	L6	`internal/gateway`	HXC-1469
Session service	L6	`internal/session`	HXC-1136
Health service	L6	`internal/health`	HXC-1484
Policy service (OPA)	L6	`internal/policy`	HXC-1135
LLM router service	L6	`internal/llm`	HXC-1470
Build service	L6	`internal/build`	HXC-904
Advisory engine	L6	`internal/advisory`	HXC-1126
Trust service	L6	`internal/trust`	HXC-1135
Federation	L7	`pkg/federation`	HXC-1359
Federation topology	L7	`pkg/fedtopology`	HXC-1372
	L7	`pkg/metrics`	HXC-1535

Component	Layer	Package	Origin
Metrics (+tier/cost/ provider)			
Distributed tracing	L7	`pkg/tracing`	HXC-1379
Grafana dashboards	L7	`pkg/grafanadash`	HXC-1551
Compliance doc-gen	L7	`pkg/compliancedoc`	HXC-1481

Maintenance

This map is kept in sync as components are added or hardened (CLAUDE-3 / §11.4.106). When you add a control-plane component, add a row here with its real package path; `pkg/archlint` will fail CI if the path does not exist, and `docs_chain` keeps the rendered HTML/PDF/DOCX exports in sync. For the broader foundation-library catalogue see `FOUNDATION_PACKAGES.md`; for the work-item registry see `HXC_REGISTRY.md`.